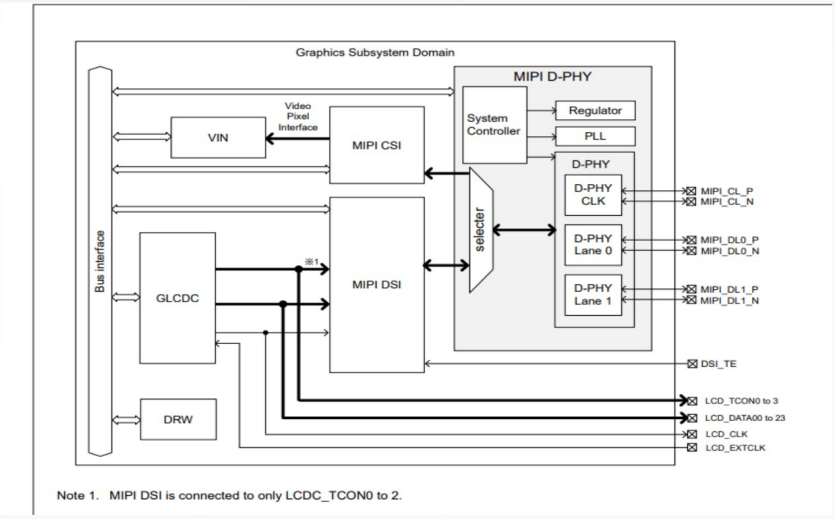
MIPI DSI

UVM-Based Controller Verification

ENGINEERING PROJECT REPORT

Prepared by: Sushant Gautam

Block Diagram



Architecture reference from the user's uploaded MIPI DSI presentation.

Prepared from supplied project material and actual EPWave simulation captures
30 October 2026

Abstract

This report documents a UVM-based verification effort for a MIPI Display Serial Interface (DSI) controller. The supplied project context describes a controller with an AHB subordinate register interface, DC-side pixel stimulus, PPI-side observation, and scoreboard infrastructure. The target active image dimensions are 1280 × 720. Four actual EPWave screenshots are embedded as the primary simulation evidence.

The screenshots show byte-valued activity on named PPI data-lane buses, long_packet and short_packet buses, and ppi_lane0_en in time windows around 13.16–13.38 µs. They do not show a clock-lane waveform, differential P/N signals, explicit LP/HS state encoding, decoded packet fields, or ECC/CRC pass/fail. Synopsys SVT AHB VIP integration is treated as in progress/unverified.

Contents

- 1. Project overview and objectives
- 2. MIPI DSI background and architecture
- 3. DUT and UVM verification environment
- 4. Stimulus and test flow
- 5. Actual simulation outputs and waveform analysis
- 6. Results, limitations and next steps
- 7. Conclusion
- Appendix: signal glossary and references

1. Project overview and objectives

The project verifies a MIPI DSI host/controller that accepts configuration accesses over an AHB subordinate interface and produces display-related traffic toward a PPI/D-PHY boundary. The documented active image dimensions are 1280 pixels per line and 720 active lines per frame.

- Exercise controller register programming through AHB.
- Provide display/pixel stimulus through a DC-side sequence/agent.
- Observe PPI lane-data and packet-related buses during simulation.
- Organize stimulus and observation using UVM agents, sequences, monitors, an environment and scoreboard infrastructure.

Evidence boundary: uploaded diagrams describe architecture; the four EPWave images are actual simulation outputs. Where compile or execution status is not established by supplied evidence, it is explicitly marked unverified.

2. MIPI DSI background and architecture

MIPI DSI is a serial display interface between a host processor/controller and a display peripheral. The supplied design material describes packet-based transport over one or more data lanes plus a clock lane, using MIPI D-PHY. The host controller performs packet generation, mode/lane management and protocol timing; the PHY handles serialization and electrical signaling.

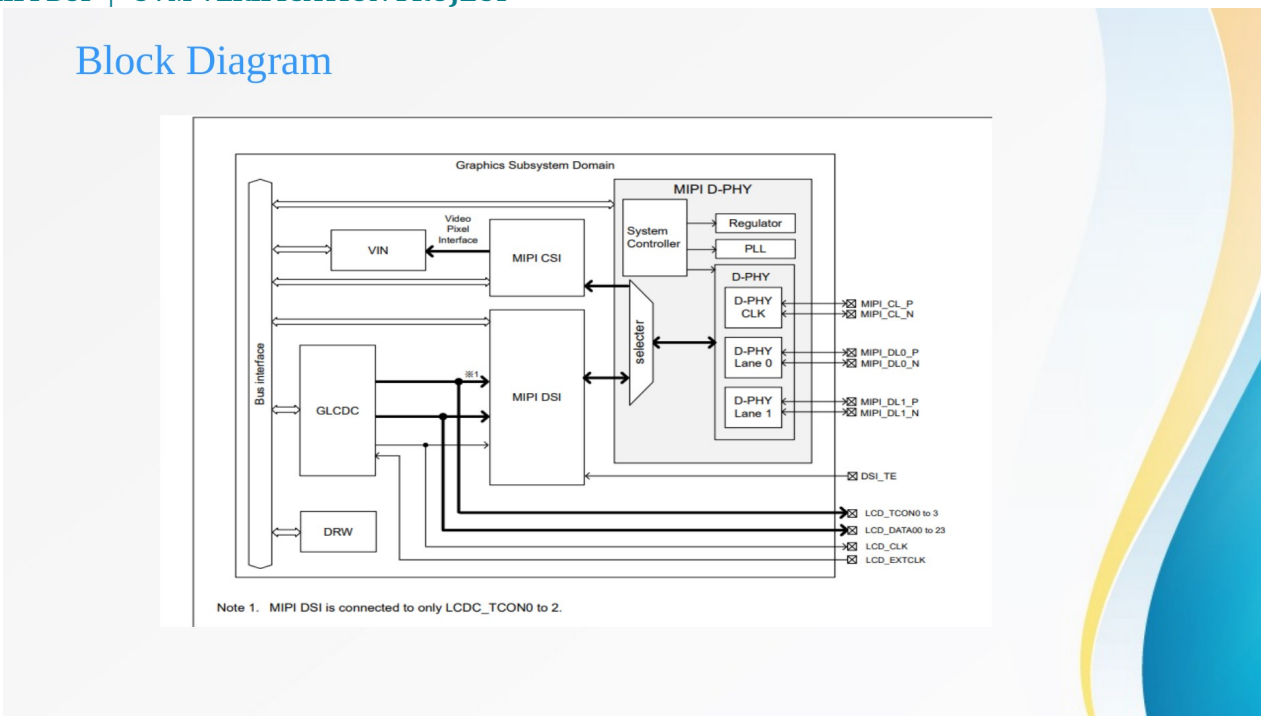


Figure 1. Original architecture diagram from the uploaded MIPI DSI presentation: host/controller subsystem, PHY and display interface.

2.1 Layered architecture

Layer / block	Responsibility
Application / pixel input	Provides pixel data and display timing/control information.
Low-level protocol / packet layer	Forms packet headers and payloads; applies packet data types and protection fields.
Lane management	Distributes or merges data across configured lanes and manages lane-level control.
D-PHY	Serializes data and provides HS/LP electrical signaling on clock/data lanes.
Display peripheral	Receives packets and reconstructs pixel or command information.

2.2 Operating modes

Mode	Purpose
Video mode	Streams pixel data using video-oriented packet sequences; supports burst and non-burst variants.
Command mode	Transfers commands and parameters to a display controller.
High-Speed (HS)	High-throughput serial transfer, typically used for pixel payloads.
Low-Power (LP)	Lower-speed control/escape communication; reverse communication is associated with data lane 0 in the supplied overview.

2.3 Short and long packets

The supplied DSI material describes short packets as fixed four-byte structures carrying a Data Identifier and two data bytes, protected by an ECC byte. Long packets contain a Data Identifier, a 16-bit Word Count, an ECC-protected header, a payload whose length is defined by Word Count, and a 16-bit checksum/CRC footer. Short packets are used for synchronization events and command/control information; long packets carry pixel payloads or larger command data.

2.4 Video timing

Horizontal timing is measured across one line; vertical timing is measured across a complete frame. HSA and VSA represent the sync pulse widths themselves, not the total blanking interval.

Parameter	Meaning
HSA	Horizontal Sync Active: horizontal sync pulse width.
HBP	Horizontal Back Porch: blanking after sync and before active pixels.
HACT	Active pixels per line; project target = 1280.
HFP	Horizontal Front Porch: blanking after active pixels.
VSA	Vertical Sync Active: vertical sync pulse width.
VBP	Vertical Back Porch: blanking lines before active image.
VACT	Active image lines; project target = 720.
VFP	Vertical Front Porch: blanking lines after active image.

HTOTAL = HSA + HBP + HACT + HFP

VTOTAL = VSA + VBP + VACT + VFP

No numeric porch or sync values were supplied as verified configuration for this DUT; none are invented here.

3. DUT and UVM verification environment

3.1 DUT AHB interface

The DSI DUT is treated as an AHB subordinate. Its documented AHB-side inputs are haddr, hwrite, hsize, hburst, htrans, hwdata and hprot. Its outputs are hresp, hready and hrdata. The DUT interface has no hsel port; testbench wiring must match the actual RTL ports.

Direction relative to DUT	Signals
Inputs	haddr, hwrite, hsize, hburst, htrans, hwdata, hprot
Outputs	hresp, hready, hrdata

3.2 UVM architecture

Component	Role
tb_top	Clock/reset generation, DUT/interface instantiation, virtual interface configuration and run_test.
AHB agent / Synopsys SVT AHB VIP	Drives register accesses; VIP migration/integration is still being validated.

DC agent	Generates display/pixel-side stimulus.
PPI interface/monitor	Observes lane-oriented data and packet-related signals.
Environment / virtual sequence	Creates and coordinates agents and sequences.
Scoreboard	Checking infrastructure; successful checks are not established by screenshots alone.

The intended test flow is reset, AHB register configuration, DC pixel stimulus, controller packet/lane generation, and PPI-side observation. The scoreboard is part of the verification architecture, but no pass result is inferred from the waveform screenshots.

4. Stimulus and test flow

4.1 Directed AHB sequence

The project sequence context lists the following directed writes. These are described as intended sequence operations; the supplied PPI screenshots do not independently prove that each AHB transfer completed.

Address	Write data
0x4000_0000	0x00000000
0x4000_0004	0x00000029
0x4000_0008	0x00000000
0x4000_000C	0x00000001
0x4000_0010	0x00000001

4.2 DC stimulus and PPI observation

The DC-side sequence is intended to provide pixel data. The PPI observation point exposes byte-wide lane buses (lane 0 through lane 3), wide packet-related buses, and a lane-enable signal. The screenshots provide a visual record of activity on these signals in the displayed time windows.

4.3 SVT AHB VIP integration status

Project history records migration from a custom AHB agent toward Synopsys SVT AHB VIP. The VIP package/API, sequencer handle, configuration and top-level signal mapping were still being checked. Complete VIP integration, successful compilation and regression completion are not claimed.

5. Actual simulation outputs and waveform analysis

The following four figures are the actual EPWave screenshots supplied by the project owner. They are embedded unchanged, with captions and cautious interpretations of only the visible signals.

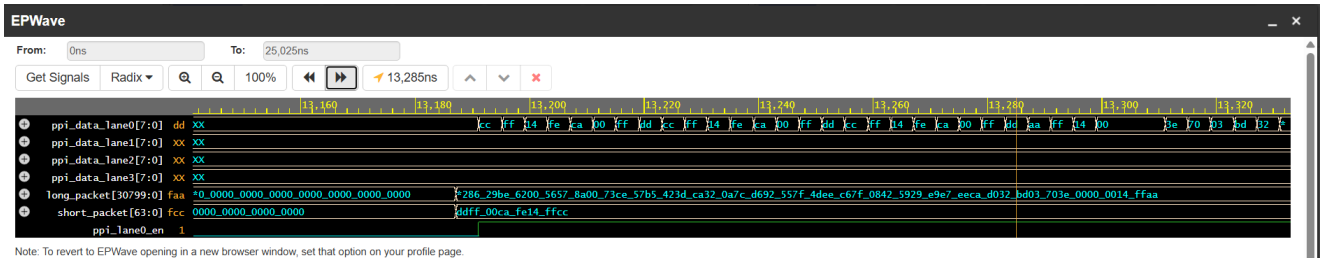


Figure 2. User EPWave output 1 — ppi_data_lane0 byte activity, long_packet, short_packet and ppi_lane0_en in the displayed approximately 13.16–13.32 µs interval.

Visible evidence: ppi_data_lane0[7:0] changes through hexadecimal byte values; long_packet[30799:0] and short_packet[63:0] are displayed as wide buses; ppi_lane0_en is visible. This image does not decode packet fields or establish protocol correctness.

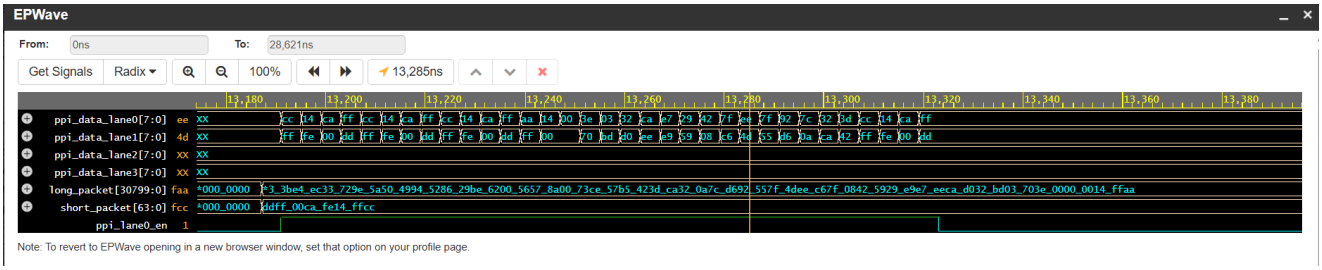


Figure 3. User EPWave output 2 — lane 0 and lane 1 show changing byte values; lane 2 and lane 3 appear X in this view. Packet buses are visible.

Visible evidence: lane 0 and lane 1 show changing hexadecimal byte values, while lane 2 and lane 3 are displayed as X in this view. The packet buses are present, but their contents are not decoded here.

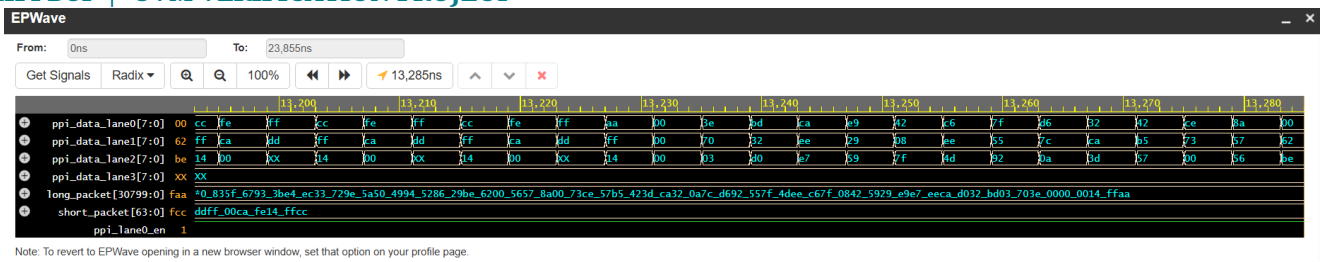


Figure 4. User EPWave output 3 — changing byte values are visible on lane 0, lane 1 and lane 2; lane 3 appears X in this capture.

Visible evidence: lane 0, lane 1 and lane 2 show changing byte values while lane 3 is X in the shown interval. This is evidence of bus activity, not proof of a valid aligned three-lane transmission.

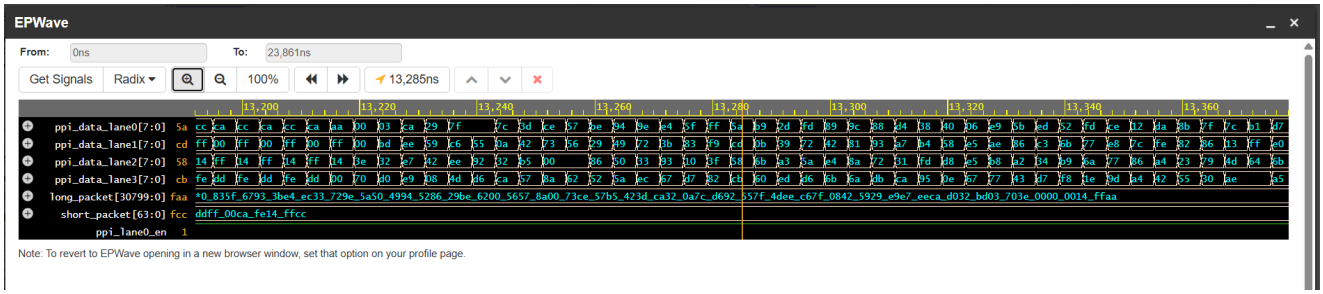


Figure 5. User EPWave output 4 — changing byte values are visible on all four named PPI data-lane buses in this capture.

Visible evidence: all four named lane buses show changing hexadecimal byte values in this capture. This does not prove lane alignment, D-PHY electrical behavior or successful display reception.

5.1 What the screenshots establish

Supported by the screenshots	Not established by these screenshots
Named PPI lane buses and changing hexadecimal values.	No clock-lane waveform or differential P/N signals.
long_packet and short_packet buses are present.	No packet decode proving Data Type, Word Count, payload meaning or ECC result.
ppi_lane0_en is visible.	No explicit LP/HS state signal or transition.
Lane activity varies across the captures.	No proof of lane alignment, throughput, full-frame completion or display output.
Waveform windows are around 13.16–13.38 µs.	No scoreboard pass, CRC/ECC pass or error-free regression.

6. Results, limitations and next steps

6.1 Results evidenced by supplied artifacts

- The project has documented MIPI DSI architecture and a UVM verification structure.
- The target active image dimensions are 1280 × 720.
- Four actual EPWave captures show packet-bus presence and changing byte values on named PPI data-lane buses.
- The screenshots show ppi_lane0_en and activity on one or more lane buses.

6.2 Limitations

- No final simulator transcript or regression summary was supplied as evidence for this report.
- The screenshots do not show AHB transactions or reset/clock waveforms.
- No decoded packet analysis or independent ECC/CRC checking result is visible.
- SVT AHB VIP integration remains in progress/unverified in the available project history.
- Short waveform windows do not establish complete frame-level operation.

6.3 Recommended next steps

- Confirm SVT AHB VIP package compile order, interface type, configuration class and master sequencer handle against the installed VIP release.
- Verify top-level mapping to the DUT's actual AHB pins without adding a nonexistent hsel port.
- Capture AHB address/control/data/response signals alongside PPI outputs to correlate register programming with lane activity.
- Add packet decoding for Data Identifier, Word Count, ECC and payload CRC/checksum; report checker status separately.
- Add assertions for reset behavior, packet sequencing, lane enable behavior and timing constraints.
- Run a repeatable regression and retain compile log, simulation log, test summary, coverage report and scoreboard results.
- Capture a complete frame/line interval and record configured timing values and lane count.

7. Conclusion

The supplied material documents a MIPI DSI controller verification effort with AHB configuration, DC-side stimulus and PPI-side observation. The four EPWave screenshots provide concrete evidence of packet-bus presence and byte-valued activity on the named PPI data buses during short simulation windows. They do not alone establish end-to-end protocol compliance or a passing regression. Finalizing SVT AHB VIP integration and adding decoded packet/checker evidence are the key next milestones.

Appendix A. Signal glossary

Signal	Description
ppi_data_lane0[7:0] … ppi_data_lane3[7:0]	Byte-wide PPI data-lane buses visible in the captures.
long_packet[30799:0]	Wide packet-related bus shown in the waveform viewer; internal field decoding is not established.
short_packet[63:0]	Short-packet-related bus shown in the waveform viewer.
ppi_lane0_en	Lane 0 enable/control signal visible in the screenshots.
haddr / hwdata	AHB subordinate address and write-data inputs.
hwrite / hsize / hburst / htrans / hprot	AHB transfer control and attributes.
hresp / hready / hrdata	AHB response, ready and read-data outputs.

References

- User-supplied MIPI DSI presentation (uploaded PPT): architecture, packet formats, modes and video timing.
- User-supplied DSI verification document (uploaded DOCX): testbench architecture and project context.

- User-supplied MIPI DSI overview slides (uploaded PPT): host/display interface, clocks, modes and timing.
- Four user-supplied EPWave screenshots (lane1.png–lane4.png): actual simulation outputs.
- MIPI Alliance DSI and D-PHY specifications; exact licensed revision used by the DUT was not identified in supplied artifacts.